

Chapter 10

玩家角色-移動

Horazon

手機程式設計

複習：上週重點

- [x] 主角有了 Capsule Collider 2D。
- [x] 主角有了 Rigidbody 2D (且不會跌倒)。
- [x] 主角不會黏牆壁 (Slippery Material)。
- [x] 建立了 `PlayerController.cs` 腳本空殼。

今天，我們要賦予他**行動力**！

本章目標

1. 理解 Unity Input Manager (輸入管理)。
2. 讀取鍵盤訊號 (`Input.GetAxis`)。
3. 使用物理速度 (`Rigidbody.velocity`) 移動角色。
4. 解決「太空漫步」問題 (角色翻面)。

輸入系統：Input Manager

Unity 幫我們整合了鍵盤、滑鼠、搖桿。

1. Edit -> Project Settings -> Input Manager。

2. 展開 Axes -> Horizontal。

3. 你會看到：

- Negative Button: `left` (左鍵), `a`。
- Positive Button: `right` (右鍵), `d`。
- Gravity / Sensitivity: 靈敏度設定。

我們寫程式時，只要呼叫 "Horizontal"，Unity 就會幫我們偵測這些按鍵。

讀取輸入 (Coding)

在 `PlayerController.cs` 的 `Update` 中：

```
float mx; // 宣告一個變數存放左右輸入

void Update()
{
    // 讀取水平輸入 (-1 ~ 1)
    mx = Input.GetAxisRaw("Horizontal");
}
```

- `GetAxis`：有加減速緩衝 (適合賽車)。
- `GetAxisRaw`：反應直接 (0 瞬間變 1)，適合 2D 動作遊戲。

施加移動 (Velocity)

有了輸入訊號 `mx` (左=-1, 不動=0, 右=1)，我們要推動剛體。
請在 `FixedUpdate` 執行物理指令！

```
void FixedUpdate()  
{  
    // 設定剛體的速度  
    // X軸 = 輸入 * 速度  
    // Y軸 = 維持原本的速度 (不要把重力歸零！)  
    rb.velocity = new Vector2(mx * moveSpeed, rb.velocity.y);  
}
```

Warning: 千萬不要寫 `new Vector2(..., 0)`，這樣角色在空中會掉不下來！

完整程式碼預覽

```
public class PlayerController : MonoBehaviour
{
    public float moveSpeed = 5f;
    public Rigidbody2D rb;
    float mx;

    void Start() {
        rb = GetComponent<Rigidbody2D>();
    }

    void Update() {
        mx = Input.GetAxisRaw("Horizontal");
    }

    void FixedUpdate() {
        rb.velocity = new Vector2(mx * moveSpeed, rb.velocity.y);
    }
}
```

實作測試 1：移動

1. 存檔，回到 Unity。
2. 按下 Play。
3. 按 A / D 或 **左右鍵**。
4. 主角應該會左右滑動！
5. **調整手感：**
 - 在 Inspector 調整 **Move Speed**。
 - 試試看 5, 8, 10，找到你覺得最順手的速度。

問題：太空漫步 (Moonwalking)

當你往左走的時候，主角的臉還是朝右邊...
這看起來像麥可傑克森在跳舞。

我們需要在往左走時，把圖片「翻過來」。

翻面邏輯 (Flipping)

有兩種方法：

1. `SpriteRenderer.flipX`：

- 只翻轉圖片。
- 缺點：如果你手上有拿槍 (子物件)，槍不會跟著翻過去。

2. `Transform.localScale`：

- 把 X 軸縮放改為 `-1`。
- **優點**：所有子物件 (槍、背包、檢測器) 一起翻轉。
- **我們採用這個方法！**

撰寫翻轉程式

在 `Update()` 裡面加入判斷：

```
void Update()
{
    mx = Input.GetAxisRaw("Horizontal");

    // 如果輸入向右 (正數)
    if (mx > 0)
    {
        transform.localScale = new Vector3(1, 1, 1);
    }
    // 如果輸入向左 (負數)
    else if (mx < 0)
    {
        transform.localScale = new Vector3(-1, 1, 1);
    }
    // 如果 mx == 0 (沒按鍵)，不動，保持原狀
}
```

實作測試 2：翻轉

1. 存檔，回到 Unity。
2. 按下 **Play**。
3. 往左走 -> 變身為左撇子。
4. 往右走 -> 變回來。
5. 觀察 Inspector 的 Transform -> Scale X 數值變化。

進階技巧：跑步功能 (Sprint)

大部分遊戲都有「按住 Shift 加速」。

1. 宣告變數：`public float runSpeed = 8f;`
2. 宣告變數：`float currentSpeed;`
3. 程式邏輯：

```
if (Input.GetKey(KeyCode.LeftShift))
{
    currentSpeed = runSpeed;
}
else
{
    currentSpeed = moveSpeed;
}

// FixedUpdate 裡改用 currentSpeed
rb.velocity = new Vector2(mx * currentSpeed, rb.velocity.y);
```

跨平台支援 (Gamepad)

你知道嗎？你寫的這段程式碼已經支援搖桿了！

- `Input.GetAxis("Horizontal")` 預設對應 Xbox/PS 手把的左類比搖桿。
- 插上手把，直接就能玩！

(這就是 *Unity Input Manager* 的強大之處)

常見問題 (Debug)

Q: 角色移動很慢？

A:

1. 檢查 `moveSpeed` 數值。
2. 檢查剛體的 `Linear Drag` (線性阻力)，空中通常設 0。

Q: 角色放開按鍵後還會滑行？

A:

1. 如果用 `GetAxisRaw` + `velocity` 直接賦值，應該是瞬間停。
2. 除非你有用 `AddForce` (施力模式)。

Q: 角色掉到地板下？

A:

檢查 Collider 是否太小，或是 Collision Detection 沒開 Continuous。

總結

今天我們完成了：

1. 讀取輸入：`Input.GetAxisRaw("Horizontal")`
2. 物理移動：`rb.velocity`
3. 角色翻轉：`transform.localScale`

主角終於聽話了！可以在場景裡自由奔跑。

下週預告

只能左右跑不夠酷。

平台遊戲的靈魂在於**跳躍** (Jump)。

下週我們要挑戰最難的一關：

- 跳躍力道。
- **地面檢測** (Ground Check)：怎麼知道我踩到地了？(防止無限二段跳)

Q & A

- 可以做俯視角 (Top-Down) 遊戲嗎？
 - 原理一樣，只是多了 Y 軸輸入 (`Input.GetAxis("Vertical")`) 去改變 `velocity.y` (要把重力關掉)。
- 為什麼不用 `transform.Translate` ？
 - 因為 `Translate` 會無視物理碰撞，可能會穿牆。

(助教巡堂協助)